

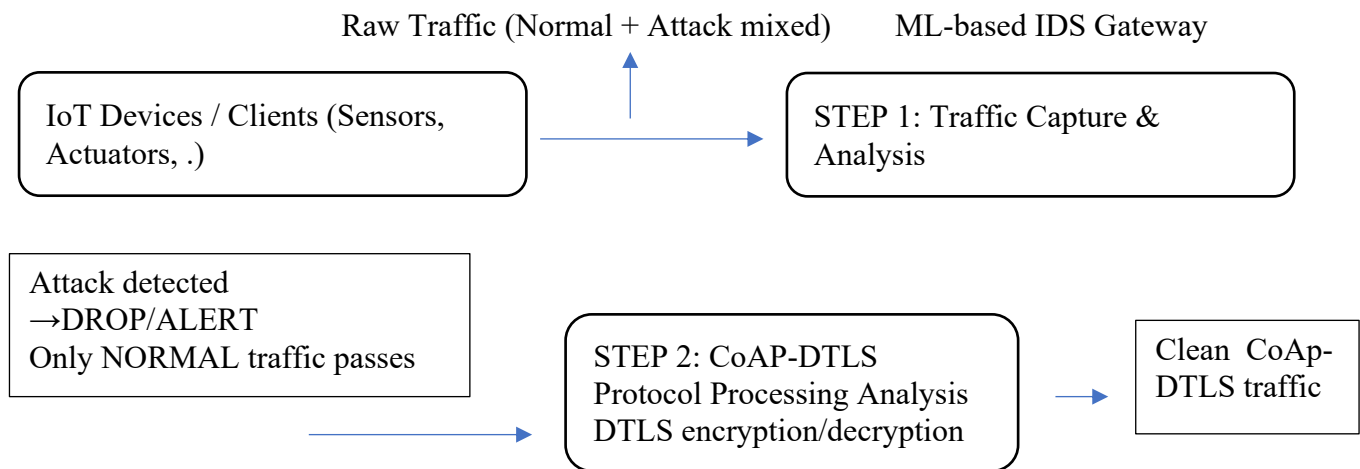
テーマ：自分のシステム環境における構成管理と脆弱性対策の改善提案

対象システムの概要

IoT 環境における二層セキュリティアーキテクチャの研究開発環境。第一層で機械学習ベースの IDS が全トラフィックを分析し攻撃を検知・遮断、第二層で正常と判定されたトラフィックのみが CoAP-DTLS プロトコルで暗号化通信を行う。現在は開発段階で、Raspberry Pi 3 台と Ubuntu Server で構成され、手動による構成管理を実施している。

1. 構成管理の現状把握

1.2 システムアーキテクチャの詳細



設計の特徴

- 第一層防御: 機械学習による異常検知（全トラフィック分析）
- 第二層防御: CoAP-DTLS による暗号化通信（正常トラフィックのみ）
- 効率性: 攻撃トラフィックに CPU 集約的な暗号化処理を行わない
- 深層防御: 二層の独立したセキュリティメカニズム

1.3 現状の管理手法: 手動管理の実態

現在、システム構成は完全に手動で管理されている。

1. デバイス設定

- Raspberry Pi へ SSH 接続して個別に設定
- Static IP アドレスを/etc/dhcpd.conf に手動記述
- CoAP サーバスクリプトを手動コピー・実行

2. Gateway 設定

- Ubuntu Server のネットワークを Netplan で手動設定
- Python 依存関係を pip install で個別インストール
- ML 学習スクリプトとモデルファイルをローカル保存

3. 変更管理

- 設定変更は個人メモ（テキストファイル）に記録
- スクリプト修正は日付付きファイル名で保存
例: gateway_v1.py, gateway_v2_20241115.py
- バージョン管理ツール（Git 等）は未使用

【部分的なスクリプト化】

一部の反復作業は簡易的に Bash スクリプト化している。

1.4 主な課題

a. 再現性の欠如

- 環境構築手順が明文化されていない
 - 新しいデバイス追加時に設定を思い出せない
 - デバイス故障時に同じ設定を再現できない
- ライブラリバージョンが記録不完全
 - requirements.txt は存在するが更新されていない

b. 変更追跡の不備

- いつ・誰が・何を変更したかが不明
 - 問題発生時に原因特定が困難
 - 以前の動作する状態に戻せない
- ML 学習パラメータの変更履歴なし
 - どのパラメータでどの精度が出たか追跡不可
 - 実験の再現性が担保されない
- 設定ファイルのバージョン管理なし
 - coap.conf, dtls.conf 等の変更履歴が不明

c. バックアップ体制の不足

- 定期的な自動バックアップなし
 - 設定ファイル消失のリスク
 - 学習済みモデルの喪失リスク
- バックアップがあっても不完全
 - 一部ファイルのみ手動コピー
- データセット管理の欠如
 - pcap ファイルが散在、命名規則なし
 - どのデータでどのモデルを学習したか不明

2. 脆弱性対策の実態分析

2.1 現状の脆弱性対策

層	現状の脆弱性対策	課題
第一層 (ML-IDS)	- Python ライブラリ: 不定期更新 - Adversarial Attack 対策: 未実装 - モデル更新頻度: 初期学習以降なし	- 依存ライブラリの脆弱性放置リスク - 敵対的攻撃に無防備 - モデル劣化 (モデルドリフト) の可能性
第二層 (CoAP-DTLS)	- DTLS 証明書: 手動更新 - CoAP ライブラリ: 半年以上更新なし - 既知の CoAP 脆弱性(CVE): 未確認	- 証明書期限切れのリスク - 古いライブラリ使用による攻撃面増加 - 脆弱性情報への追従不足
層間連携 (統合セキュリティ)	- ML-IDS バイパス攻撃の検証なし - DTLS 層への直接攻撃シミュレーションなし	- 2 層間の防御境界が不明確 - システム全体の攻撃シナリオに対処できない

攻撃シナリオ:

1. ML-IDS を正常トラフィックと誤認させる
2. CoAP-DTLS 層に悪意のあるペイロード送信
3. 暗号化後にバックエンドを攻撃

2.2 CVE / JVN 情報の確認状況

観点	現状	課題
Python ライブラリ (ML-IDS)	定期的な CVE 確認なし	依存パッケージの脆弱性を見落とす可能性
CoAP ライブラリ (libcoap / aiocoap)	CVE/JVN 情報を確認していない	既知脆弱性に気づかず運用を継続してしまう
DTLS (mbedTLS / tinyDTLS)	情報収集プロセスなし	暗号アルゴリズムや実装バグの影響を受けやすい
Raspberry Pi OS / Ubuntu Server	手動確認のみ	OS レベルの脆弱性対応が遅れる

2.3 パッチ適用・ライブラリ更新の頻度

層	現状	課題
ML-IDS (Python 環境)	ライブラリ更新は不定期、必要時のみ	セキュリティパッチ適用が遅れやすい
ML モデル更新	初期学習後ほぼ更新なし	モデル劣化・検知率低下に気づけない
CoAP ライブラリ	半年以上更新なし	古い実装を使い続けるリスク増大

層	現状	課題
DTLS 証明書	有効期限切れ前に手動更新	自動管理がなく、期限切れの可能性
OS のセキュリティ更新	気づいたときに手動適用	一貫した更新管理がない

2.4 既知の課題・放置されているリスク

分類	内容	想定されるリスク
ML-IDS	ライブラリ更新遅延	既知の CVE が残存
ML-IDS	モデル更新なし	検知率低下に気づけない（モデルドリフト）
CoAP-DTLS 層	CoAP ライブラリ脆弱性未確認	攻撃者に既知 CVE を悪用される可能性
	DTLS 証明書手動管理	有効期限切れによる通信停止
層間連携	ML-IDS バイパス攻撃の検証なし	システム全体で統合防御が機能しない

3. 改善提案と自動化の構想

1. 構成管理の完全自動化による運用効率の向上

Git、Docker、Ansible を統合的に活用することで、環境構築からデバイス設定までの一連の作業を自動化できると考える。これにより、従来手動で行っていた環境差異の解消、設定反映漏れ、依存関係の不整合といった課題が根本的に解決される。

Git: すべての設定・コード変更を履歴管理し、再現性と監査性を確保

Docker: 開発・検証・実験環境を完全にコンテナ化し、どのマシンでも同一環境を即時構築

Ansible: Raspberry Pi やサーバ構成を自動的にデプロイし、設定の一貫性を維持

2. 脆弱性対策の継続化・高度化

本提案では、依存ライブラリ管理、脆弱性スキャン、証明書管理を自動化することで、セキュリティ体制の継続的な強化を実現する。

週次自動スキャンの実行（Trivy、OpenVAS 等）

Pull Request 時の自動セキュリティチェック

Dependabot / Renovate によるライブラリ自動更新

これにより、重大な脆弱性の放置、証明書の期限切れ、CoAP/DTLS ライブラリの旧バージョンによる攻撃リスクが大幅に低減する。

研究用システムとはいえ、近年の IoT 環境攻撃の高度化を踏まえると、継続的な脆弱性管理は不可欠である。

3. 実現に向けた優先順位（例）

優先度 高:1Git による設定の一元管理__Docker を用いた環境の標準化

DTLS 証明書の自動更新スクリプトの整備

優先度 中:2Ansible による Raspberry Pi の構成管理自動化週次脆弱性スキャンの
自動化 Dependabot / Renovate の導入

まとめ

本提案は、ML-IDS と CoAP-DTLS システムのセキュリティや運用の効率を改善することを目的としています。今後は、提案した自動化や改善策を優先順位に沿って少しずつ実施し、実験環境の安定性や研究成果の信頼性を高めていきたいと考えています。

References:

- 1.Sara Alayda, Randa Ahmed, Data Protection in IoT using CoAP based on Enhanced DTLS,Journal of IoT Security, 2020.
- 2.National Vulnerability Database (NVD), CVE Details, <https://nvd.nist.gov>
- 3.Dependabot Documentation, GitHub, <https://docs.github.com/en/code-security/supply-chain-security/keeping-your-dependencies-up-to-date-automatically>
- 4.Ansible Documentation, <https://docs.ansible.com>
- 5.Docker Documentation, <https://docs.docker.com>